

Encode URL Encode a URL to a percent-encoded form, excluding already-encoded sequences. Installation <pre>npm install encodeurl</pre> API <pre>var encodeUrl = require('encodeurl')</pre> 1	encodeURIComponent(url) Encode a URL to a percent-encoded form, excluding already-encoded sequences. This function accepts a URL and encodes all the non-URL code points (as UTF-8 byte sequences). It will not encode the “%” character unless it is not part of a valid sequence (%20 will be left as-is, but %foo will be encoded as %25foo). This encode is meant to be “safe” and does not throw errors. It will try as hard as it can 2	to properly encode the given URL, including replacing any raw, unpaired surrogate pairs with the Unicode replacement character prior to encoding. Examples Encode a URL containing user-controlled data <pre>var encodeUrl = require('encodeurl') var escapeHtml = require('escape-html')</pre> 3	<pre>http.createServer(function onRequest (req, res) { // get encoded form of inbound url var url = encodeURIComponent(req.url) // create html message var body = '<p>Location ' + escapeHtml(url) + ' not found</p>' // send a 404</pre> 4
8 <pre>res.setHeader('Location', location)) , 'utf-8')) String(Buffer.byteLength(body, res.setHeader('Content-Length', html; charset=UTF-8') res.setHeader('Content-Type', 'text/ res.statusCode = 301 // send a 301 escapeHtml(location) + '</p>' + site: var body = '<p>Redirecting to new</pre>	7 <pre>// create html message encodeURIComponent(url.format(href)) var location = // create location header href.slashes = true href.protocol = 'https:' href.host = 'localhost' // set new host for redirect</pre>	9 <pre>http.createServer(function onRequest (req, res) { // parse inbound url var href = url.parse(req) var url = require('url') var escapeHtml = require('escape-html') var encodeUrl = require('encodeurl')</pre> Encode a URL for use in a header field	5 <pre>res.statusCode = 404 res.setHeader('Content-Type', 'text/html; charset=UTF-8') res.setHeader('Content-Length', String(Buffer.byteLength(body, 'utf-8')) res.end(body, 'utf-8') })</pre>
<pre>res.end(body, 'utf-8') })</pre> Similarities This function is <i>similar</i> to the intrinsic function encodeURI. However, it will not encode: - The \, ^, or characters - The % character when it’s part of a valid sequence [and] (for IPv6 hostnames) 9	Replaces raw, unpaired surrogate pairs with the Unicode replacement character As a result, the encoding aligns closely with the behavior in the WHATWG URL specification. However, this package only encodes strings and does not do any URL parsing or formatting. It is expected that any output from new URL(url) will not change when used with this package, as the output has already been encoded. Additionally, if we were to encode before new URL(url), we do not expect the 10	before and after encoded formats to be parsed any differently. Testing <pre>\$ npm test \$ npm run lint</pre> References RFC 3986: Uniform Resource Identifier (URI): Generic Syntax WHATWG URL Living Standard 11	License MIT 12
91	11	14	13